

prog2 lab B1

Esercizi array e strutture

Esercizio su struct (1.2) stringhe dinamiche

vorrei creare una stringa dinamica di capacita' modificabile, da utilizzare per raccogliere stringhe arbitrarie in una unica.
per semplicita' diamo un esempio del' file di include nella prossima slide.
per il testing realizziamo una funzione ad hoc.

NB in linguaggi a piu' alto livello esistono delle librerie di base che realizzano questo comportamento, in C si parte quasi sempre da 0.

```
#include "dynString.h"
```

Esercizio su stringa dinamica

```
typedef struct {  
    char* content; // allocato dinamicamente  
    int capacity; // max numero caratteri  
    int last; // primo posto libero dove scrivere  
    // int dynamic; // e' stata allocata???  
} dynString, *dynStringPtr;
```

```
dynStringPtr creaDynString(int capacity); // crea  
void append(dynStringPtr s, char* s2); // concatena  
char* toCharStar(dynStringPtr p); // prendi solo la stringa  
void cancellaDynStr(dynStringPtr p); // libera memoria  
void resetDynStr(dynStringPtr p); // azzera il contenuto  
int testDynString(); /// testa se funziona
```

Esercizio su struct (1.2) testing

questa struct dovrebbe semplificare l'uso nei punti di utilizzo.

la funzione importante e' append():

- calcola lunghezza stringa da aggiungere
- se non c'e' spazio sufficiente, realloca per renderlo possibile
- finalmente copia al posto giusto e aggiorna il primo posto libero

la funzione tocharstar(), a parte restituire il puntatore,

deve anche mettere '\0' per terminarla e renderla compatibile con le normali stringhe. notare che appendere '\0' potrebbe dare out of bounds!!!

occorre quindi testare come nel caso normale, (a questo punto si duplicherebbe il codice, ma noi lo fattorizzeremo.

Esercizio stringa dinamica

creare una stringa allocata dinamicamente che cresce di dimensione quando necessario, e che realizzi le funzioni definite in **dynString.h**

// un problema sorge quando forniamo il contenuto a un chiamante: creare una copia o no? se non copiamo il chiamante puo' modificare il nostro contenuto!!! potremmo sempre creare due funzioni, una duplica e una no.

// generalizzazione: volendo si potrebbe anche usare delle dynString preallocate, (ma con capacity ==0 inizialmente).
starebbe pero' all'utilizzatore fare la free() del contenuto

Esercizio su struct (2)

modellare questa struttura:

- un pollaio contiene tante galline (quante voglio)
- ogni gallina ha un nome (ammessi sinonimi)
- ci sono tante uova, che non hanno un nome ma un ID (intero progressivo)
- voglio associare ogni uovo ad una gallina e viceversa

devo quindi aggiungere delle strutture dati opportune

Esercizio su struct 2

scrivere le seguenti funzioni:

- aggiungi gallina(pollaio *p, char *nome);
- deponiuovo(char * nomegallina) // vedi codice piu' avanti e crea un nuovo id univoco
- uovo * preleva(int amount) // preleva dal pollaio le N uova piu' vecchie
- gallina* cercaGallina(uovo*u); // query
- uovo * recuperaUova(gallina* g); // query

NB: i tipi sono approssimativi, potrebbero mancare attributi, a volte i puntatori potrebbero essere passati per referenza

Esercizio su struct 2

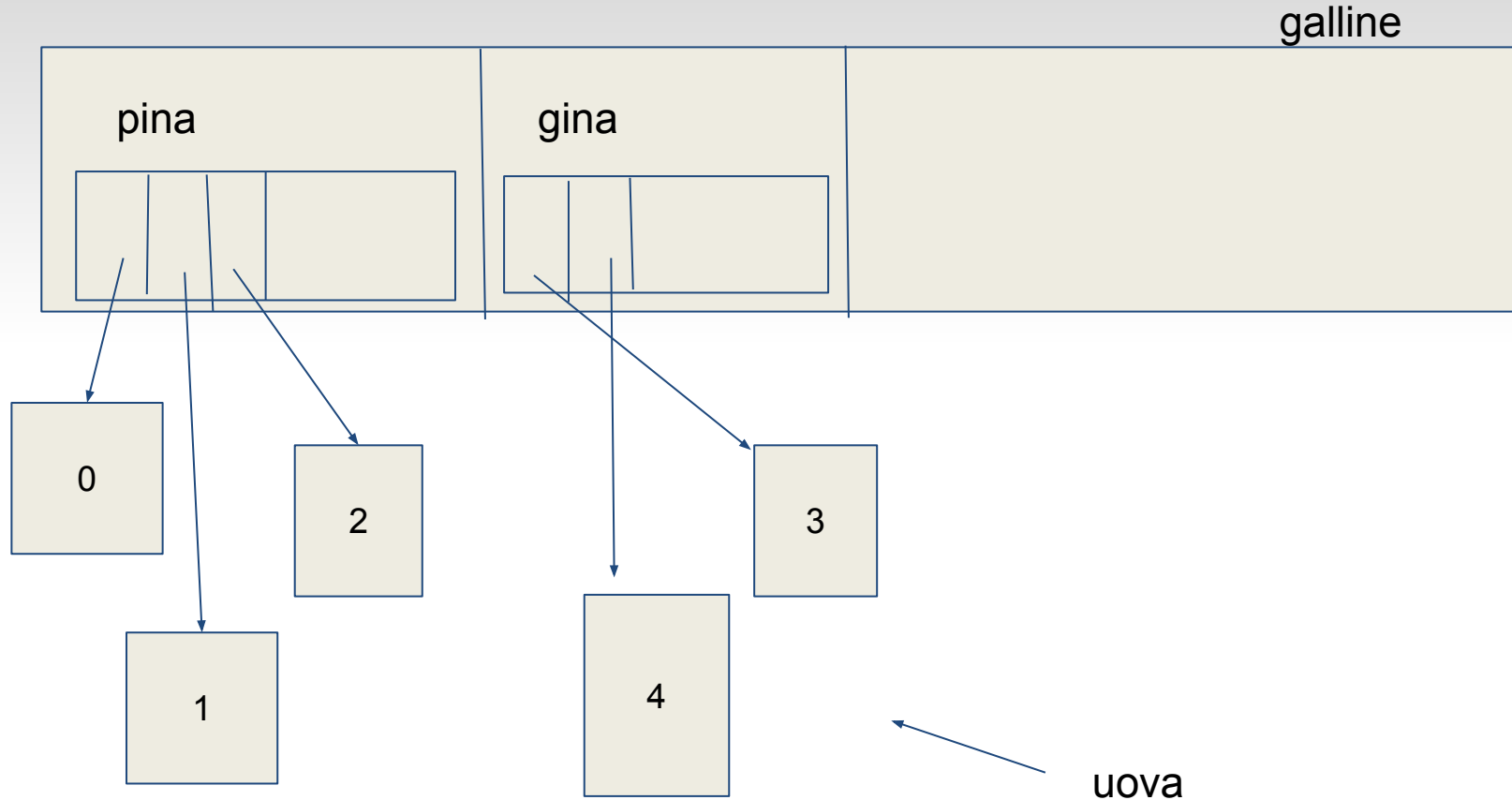
possiamo fare in diversi modi (a parte il pollaio):

creiamo due typedef gallina e uovo, poi:

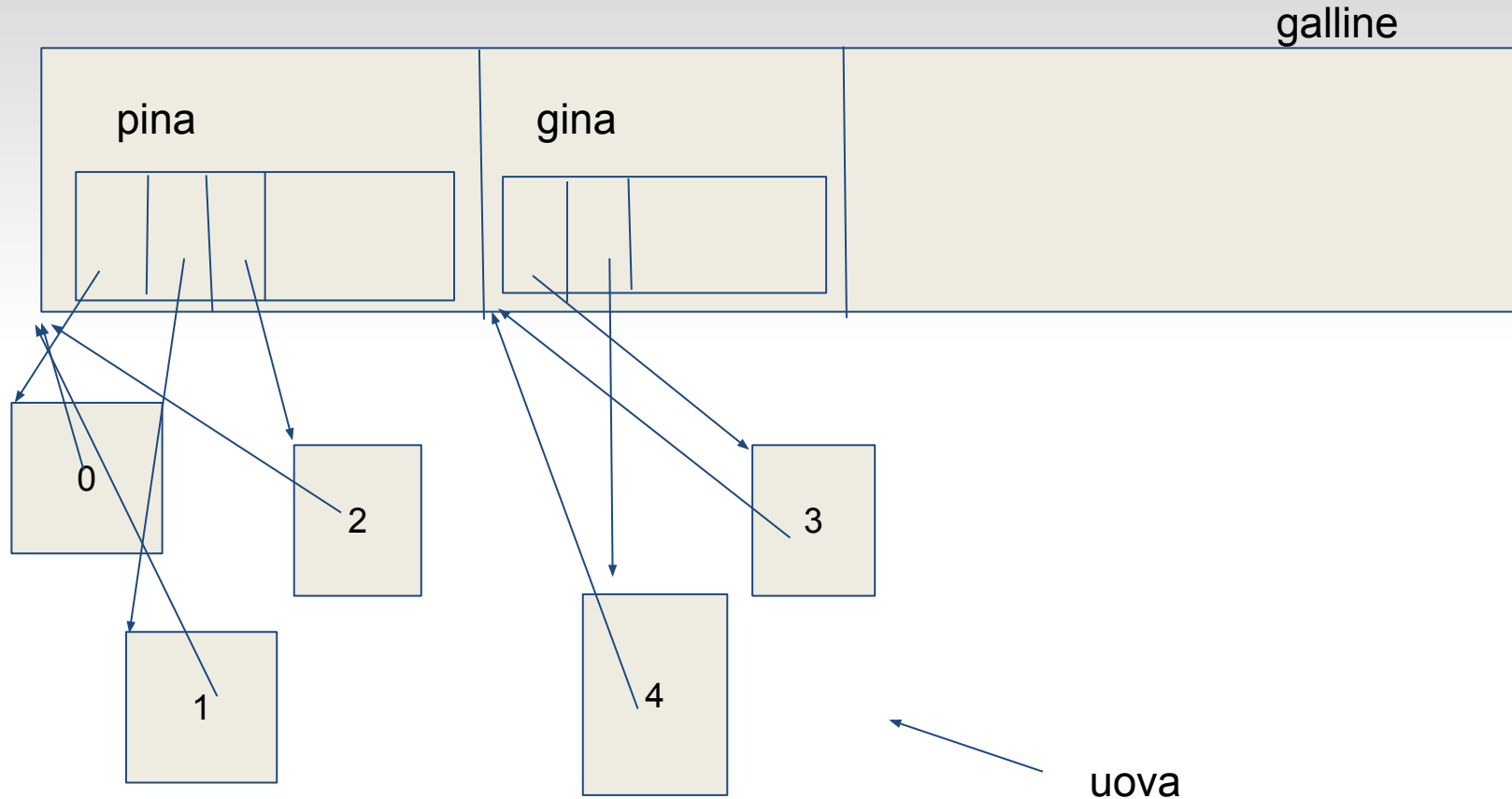
1. ogni gallina contiene un array di uova , l'uovo non conosce la gallina
2. simile a 1 ma uovo conosce gallina
3. simile a 2 ma tutte le uova sono in un array unico
4. come 3 ma gallina e uovo non si conoscono, bisogna associarle con una struttura dati indipendente

le mie soluzioni solo esercizi di modellazione, vediamo pro e contro rispetto alle operazioni richieste

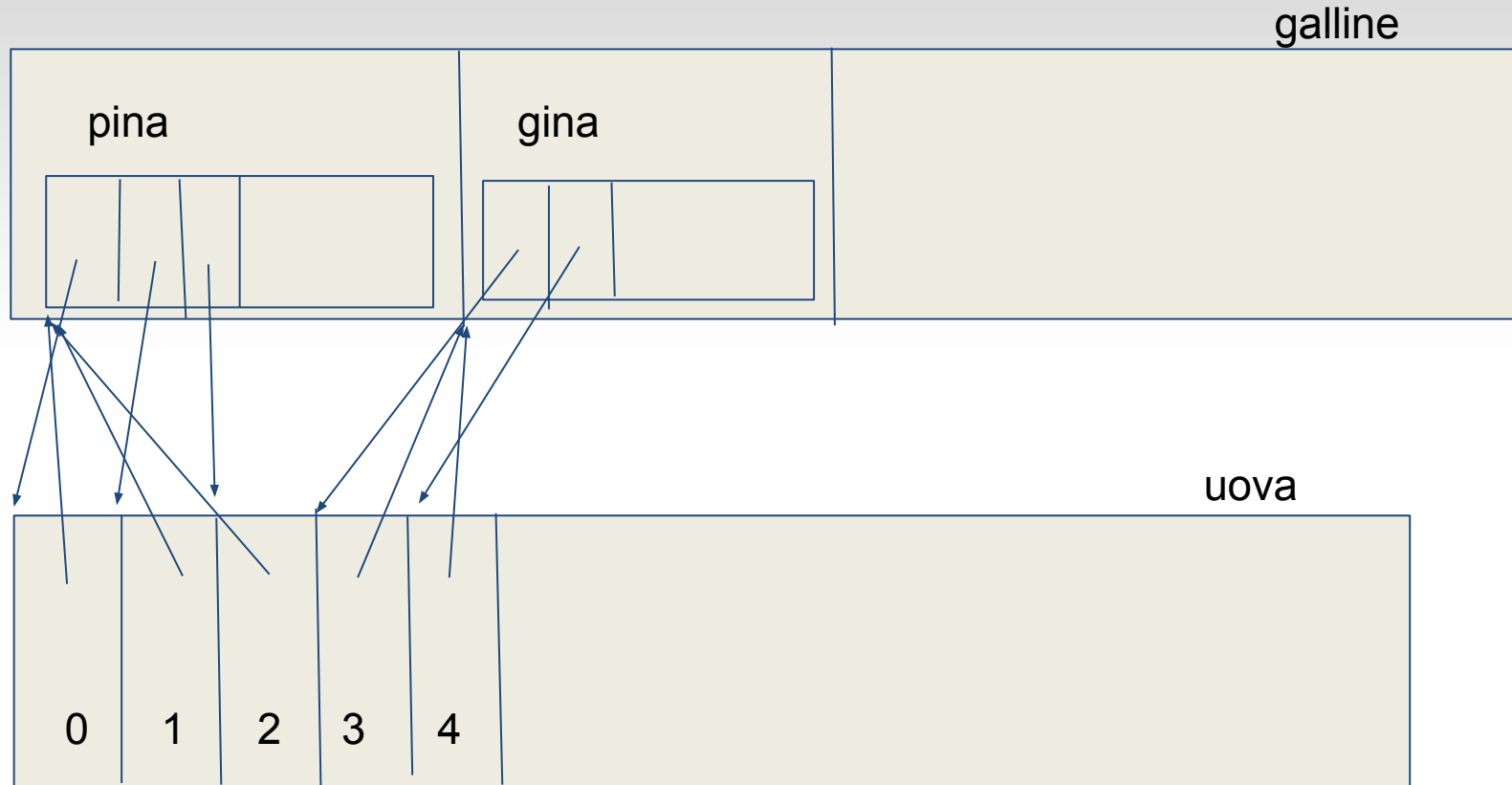
Esercizio su struct 2 soluzione 1



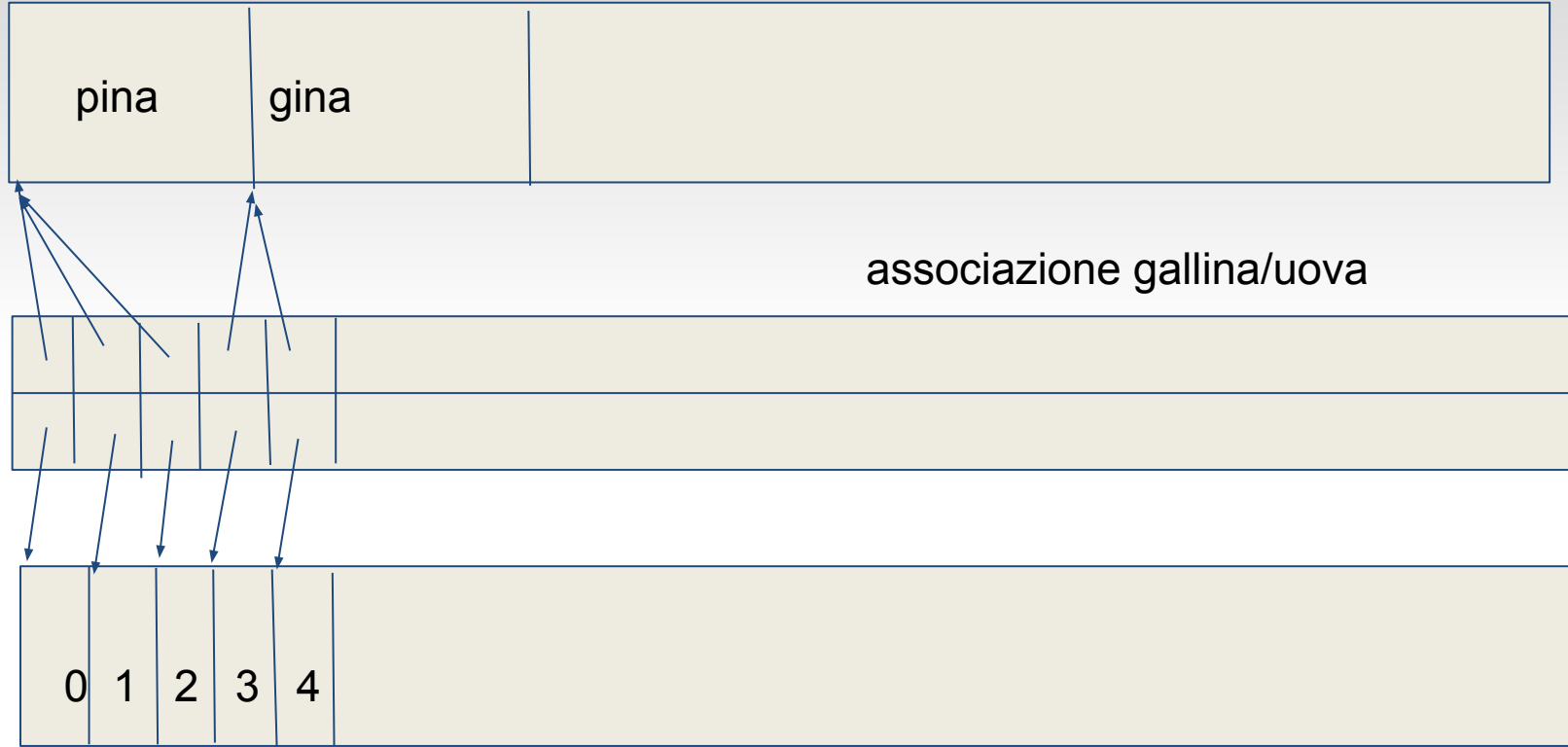
Esercizio su struct 2 soluzione 2



Esercizio su struct 2 soluzione 3



Esercizio su struct 2 soluzione 4



confronto

sol	aggiungiGallina	deponi	preleva	cercagallina	recuperauova
1	1	1	~	~	1
2	1	1	~	1	1
3	1	1	1	1	1
4	1	1	1	1	~

Esercizio su struct 2

- come occupazione di spazio le soluzioni non sono molto differenti
- la soluzione 4 svincola uovo e gallina, che diventa struct **senza** parti dinamiche
- Se per caso (ma non lo raccomando!) volessimo risparmiare spazio, potremmo trasformare i puntatori in **int** . quindi i riferimenti non sarebbero gli indirizzi di memoria ma gli indici nei due array.
- sarebbe a dire che la gallina al posto 13 ha deposto le uova al posto 11,13,22...
- Servirebbe? se $\text{sizeof}(\text{int}) < \text{sizeof}(\text{void}^*)$ si, ma in realta' occorrerebbe fare una conversione apposita, cioe' ad esempio memorizzare 2 int in un long.. cosa che si faceva quando la memoria era poca....

Esercizio su struct 2

per testing create il pollaio con 4 galline, aggiungete 20 uova e prelevatene 10

per simulare la deposizione delle galline traete spunto dal seguente codice, che usa la funzione rand() per generare un numero (pseudo) casuale

```
#include <stdlib.h>
#include <stdio.h>
```

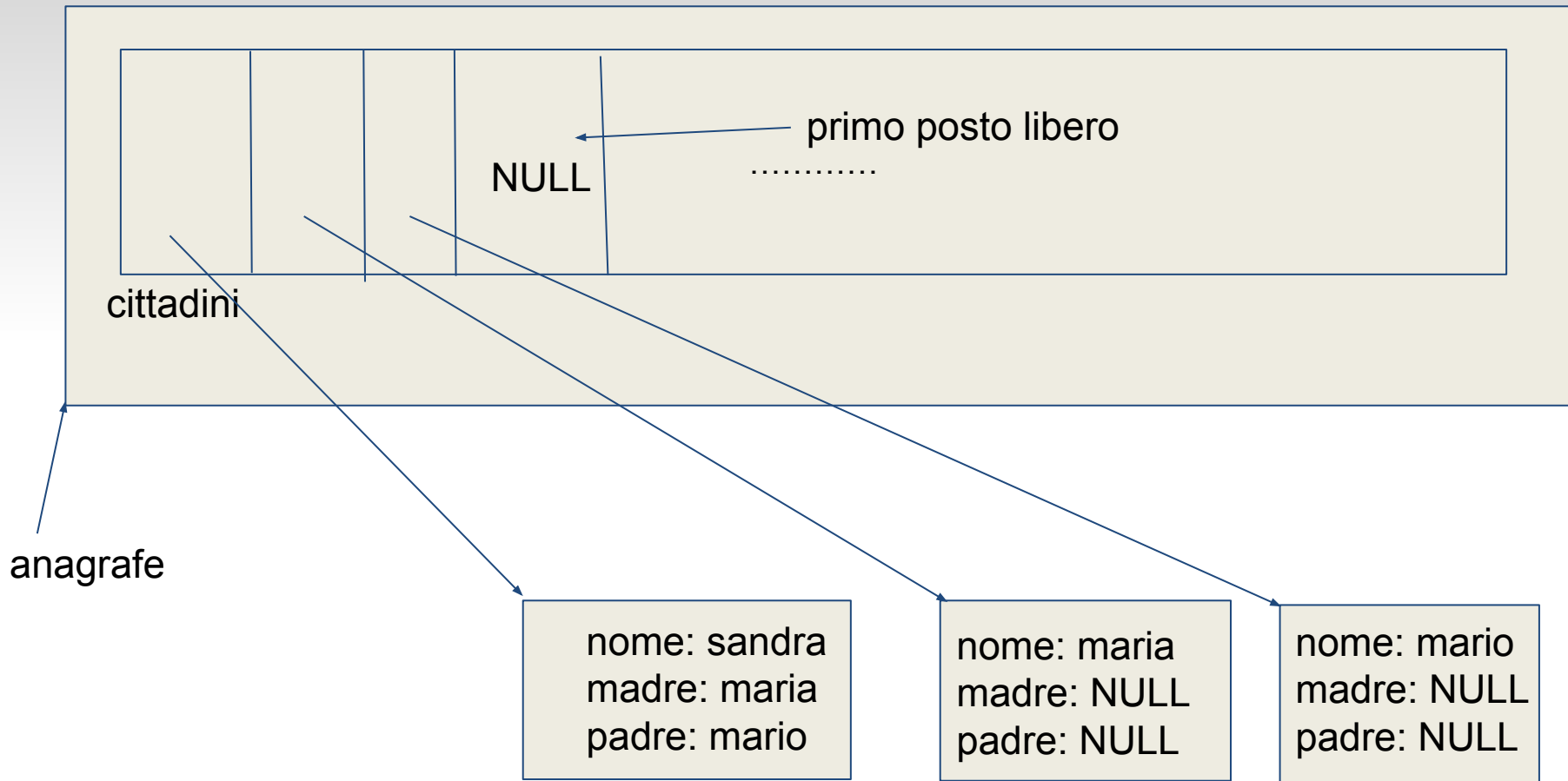
```
void crea(){
    srand(1000); // seed iniziale per ripetibilita'
    for (int i =0; i < 10; i++){
        printf("%d\n",rand()%4); // genera un intero casuale (0..3)
    }
}
```

Esercizio su struct (2)

creare delle strutture dati per modellare una anagrafe:

- delle persone descritte da 3 stringhe:
nome, madre, padre
- l' anagrafe e' un elenco di persone

Esercizio su struct (1)



Esercizio su struct (1)

implementare le seguenti funzioni:

creapersona(..)

creanagrafe(..)

aggiungipersona(..) // volendo in ordine alfabetico ?, onomimi?

cercapersona(..) // esiste dentro anagrafe

getmadre(), getpadre()

sorellafratello(..) // trovare parentela

opzionalmente create la struttura di supporto per il **testing** (esercizio 1.2):

da_persona_a_stringa(..)

da_stringa_a_persona(..)

da_anagrafe_a_stringa(..)

da_stringa_a_anagrafe()

Esercizio su struct (1)

come modellare i dati?

- le persone si possono modellare come struct
- l'anagrafe, in una prima versione potrebbe essere un array di persone
- ma vedremo che magari dovremo aggiungere altre proprietà (es la capacità) anziché usare costanti a compile time, per cui diventerà una struct anch'essa
- come fare a sapere la capacità dell'anagrafe?

Esercizio su struct (1)

in particolare, dato un array, dobbiamo conoscerne la dimensione.

non esiste nessuna funzione standard a tal fine, dobbiamo tenerne traccia noi ; potremmo creare un array contenuto in una struct, con un int che la misura:

→ `typedef struct {char * myarray; int size;} sized_array_of_char;`

capire se un posto e' libero od occupato: occorre mettere un elemento speciale

per i puntatori NULL, per altri tipi occorre valutare (es -1, '\0' ..) oppure segnare l'ultimo posto occupato (aggiungendo un attributo last):

→ `typedef struct {char * myarray; int size; int last;} sized_array_of_char;`

Esercizio su struct (1)

limitazioni:

questa struttura non e' molto flessibile: va bene finche' aggiungo persone in ordine di arrivo

- ma che succederebbe se cancellassi un elemento?
- e se volessi inserire in base a un ordinamento (es per nome)?

questi problemi si risolvono usando altre strutture dati che vedremo in seguito

definiamo ora esattamente i prototipi delle funzioni.

Esercizio su struct (1)

versione 1:

abbiamo una quantita' max fissa di persone
non permettiamo omonimi, e lo controlliamo

versione 2:

- se la capacita' attuale non e' sufficiente, si raddoppia lo spazio

versione 3:

aggiungiamo cognome ed altro per permettere omonimi
che fare se inserisco un nome di antenato che non
esiste?